

PROJETO INTEGRADOR

DOCUMENTO DE PROJETO DE SOFTWARE

Sistema Soluções Engenharia C&F

Christian Eric Barrantes Briceño

1. Identificação e visão geral

Item	Descrição
Nome do software	Soluções Engenharia C&F
Tipo	Sistema de apoio à gestão de engenharia
Versão documentada	Projeto desktop refatorado para reaproveitamento em aplicação web
Desenvolvedor	Christian Eric Barrantes Briceño
Plataforma atual	Java SE 17, NetBeans, JDBC e MySQL
Repositório	https://github.com/ChrisEricB/SolucoesEngenhariaCFRefatorado

O sistema centraliza informações relacionadas a projetos de engenharia, auditorias e não conformidades. Sua finalidade é apoiar o registro, o acompanhamento e a consulta dessas informações por profissionais com diferentes perfis de acesso.

2. Problema e objetivos

2.1 Problema

A gestão de projetos, auditorias e não conformidades pode ficar dispersa em documentos, planilhas e comunicações separadas. Essa situação dificulta a rastreabilidade, o acompanhamento de prazos, a definição de responsáveis e a consulta do histórico.

2.2 Objetivo geral

Desenvolver um sistema que permita organizar e consultar dados de projetos, auditorias e não conformidades em uma base centralizada, com autenticação de usuários e integração com banco MySQL.

2.3 Objetivos específicos

- Autenticar usuários ativos por e-mail e senha.
- Cadastrar, listar, alterar e excluir projetos.
- Registrar e acompanhar auditorias ligadas aos projetos.
- Registrar e acompanhar não conformidades e seus prazos de correção.
- Preservar a integridade dos dados por meio de validações.
- Separar regras de negócio, persistência e interface para permitir evolução web.

3. Escopo

3.1 Funcionalidades incluídas

- Gerenciamento básico de usuários e autenticação.

- Gerenciamento de projetos, incluindo cliente, responsável, datas, orçamento e status.
- Gerenciamento de auditorias internas e externas.
- Gerenciamento de não conformidades, gravidade, causa raiz, prazo e situação da correção.
- Persistência das informações em banco MySQL.
- Testes funcionais pelo método main().

3.2 Itens fora do escopo desta versão

- Interface web completa e publicação em servidor.
- Recuperação automática de senha por e-mail.
- Controle detalhado de permissões por operação.
- Armazenamento em nuvem de relatórios e documentos.
- Testes automatizados com framework JUnit.

4. Usuários do sistema

Perfil	Responsabilidade esperada
Administrador	Gerenciar informações gerais, usuários e acesso completo ao sistema.
Gerente	Acompanhar projetos, auditorias, indicadores e responsáveis.
Consultor	Participar de projetos e auditorias e registrar informações técnicas.
Engenheiro	Executar atividades de projeto e acompanhar ações corretivas.
Cliente	Perfil associado aos projetos e às consultas permitidas pela futura aplicação.

5. Requisitos funcionais

Código	Requisito funcional
RF01	O sistema deve autenticar usuários ativos por e-mail e senha.
RF02	O sistema deve permitir cadastrar, listar, buscar, atualizar e excluir usuários.
RF03	O sistema deve impedir o cadastro de e-mails duplicados.
RF04	O sistema deve permitir cadastrar, listar, buscar, atualizar e excluir projetos.
RF05	O sistema deve vincular cada projeto a um responsável e, opcionalmente, a um cliente.
RF06	O sistema deve permitir cadastrar, listar, buscar, atualizar e excluir

Código	Requisito funcional
	auditorias.
RF07	O sistema deve listar as auditorias de um projeto específico.
RF08	O sistema deve permitir cadastrar, listar, buscar, atualizar e excluir não conformidades.
RF09	O sistema deve listar as não conformidades de um projeto específico.
RF10	O sistema deve registrar o prazo, o responsável e o status de correção da não conformidade.
RF11	O sistema deve persistir os dados no banco solucoes_engenharia_cf.
RF12	O sistema deve informar erros de validação ou persistência sem ocultar a falha.

6. Requisitos não funcionais

Código	Requisito não funcional
RNF01	A aplicação deve utilizar Java SE 17.
RNF02	A persistência deve utilizar JDBC e MySQL.
RNF03	A senha do banco não deve ser publicada no código-fonte.
RNF04	As senhas dos usuários devem ser comparadas com hash SHA-256 gerado pelo MySQL.
RNF05	A arquitetura deve separar modelo, regras de negócio, contratos de persistência e implementação JDBC.
RNF06	Os recursos JDBC devem ser fechados por try-with-resources.
RNF07	Os valores monetários devem utilizar BigDecimal.
RNF08	O projeto deve ser versionado com Git e disponibilizado no GitHub.
RNF09	O projeto deve ser portátil, contendo o connector JDBC e o dump do banco.
RNF10	O código deve ser preparado para reaproveitamento em uma aplicação web.

7. Regras de negócio

7.1 Usuários

- O nome é obrigatório e deve possuir pelo menos três caracteres.
- O e-mail é obrigatório, deve possuir formato válido e não pode estar associado a outro usuário.
- A senha é obrigatória e deve possuir pelo menos seis caracteres nos cadastros e atualizações.
- Somente usuários ativos podem ser autenticados.
- O tipo do usuário é obrigatório.

7.2 Projetos

- O nome é obrigatório e deve possuir pelo menos três caracteres.
- A data de início é obrigatória.
- A data de término não pode ser anterior à data de início.
- O orçamento não pode ser negativo.
- O status e o responsável são obrigatórios.
- O cliente pode ficar sem preenchimento quando não houver vínculo definido.

7.3 Auditorias

- A auditoria deve estar vinculada a um projeto e a um auditor responsável.
- O tipo e a data agendada são obrigatórios.
- A data de realização não pode ser anterior à data agendada.
- Um resultado somente pode ser registrado quando existir data de realização.

7.4 Não conformidades

- A descrição, o projeto, a gravidade, o responsável e o prazo são obrigatórios.
- As gravidades aceitas são Baixa, Média, Alta e Crítica.
- Os status aceitos são Registrada, Em Correção, Corrigida, Verificada e Fechada.
- Quando o status indicar correção, verificação ou fechamento, a data de correção deve ser informada.
- A vinculação com uma auditoria é opcional.

8. Modelo de domínio e dados

8.1 Entidades principais

Entidade	Atributos principais	Relacionamentos
Usuario	id, nome, e-mail, senha, tipo, ativo, datas de criação e atualização	Pode ser cliente, responsável de projeto, auditor ou responsável por correção.
Projeto	id, nome, descrição, cliente, datas, orçamento, status, responsável	Possui auditorias e não conformidades.

Entidade	Atributos principais	Relacionamentos
Auditoria	id, projeto, tipo, datas, auditor responsável e resultado	Pertence a um projeto e pode originar não conformidades.
NaoConformidade	id, auditoria opcional, projeto, descrição, causa raiz, gravidade, prazo, status e correção	Pertence a um projeto e pode estar vinculada a uma auditoria.

8.2 Banco de dados

O banco utilizado possui o nome `solucoes_engenharia_cf`. O arquivo `database/solucoes_engenharia_cf_dump.sql` permite recriar sua estrutura e dados de demonstração.

Além das quatro tabelas centrais, o banco inclui tabelas de apoio para normas, relacionamento entre projeto e norma, checklists, itens de checklist, documentos, etapas, alertas e evidências. As tabelas identificadas são: `usuario`, `projeto`, `auditoria`, `naoconformidade`, `norma`, `projeto_norma`, `checklist`, `itemchecklist`, `documento`, `etapaprojeto`, `alerta` e `evidenciachecklist`.

9. Arquitetura e tecnologias

9.1 Organização em camadas

Camada/pacote	Descrição
model	Entidades sem dependência de interface ou de JDBC.
service	Regras de negócio, validações e normalização dos dados.
repository	Abstrações das operações de persistência.
repository.jdbc	SQL, PreparedStatement, ResultSet e mapeamento dos objetos.
config	Configuração centralizada da conexão.
app	Composição das dependências e execução dos testes.

9.2 Tecnologias

- Java SE 17
- Apache NetBeans IDE
- Java Database Connectivity (JDBC)
- MySQL
- MySQL Connector/J 9.4.0
- Git e GitHub

10. Casos de uso principais

Caso de uso	Fluxo resumido
UC01 - Autenticar usuário	Usuário informa e-mail e senha; o serviço valida os campos; o repositório procura um usuário ativo com o hash correspondente; o sistema retorna o usuário ou informa credenciais inválidas.
UC02 - Cadastrar projeto	Usuário informa os dados; o ProjetoService valida nome, datas, orçamento, status e responsável; o repositório grava e retorna o ID gerado.
UC03 - Registrar auditoria	Usuário seleciona projeto, tipo, data e auditor; o serviço valida; o repositório registra a auditoria.
UC04 - Registrar não conformidade	Usuário descreve o problema, escolhe gravidade, responsável e prazo; o serviço valida o status e a correção; o repositório grava o registro.
UC05 - Consultar informações	O sistema utiliza os serviços para listar usuários, projetos, auditorias e não conformidades.

11. Configuração e execução

1. Instalar Java 17, Apache NetBeans e MySQL.
2. Criar o banco solucoes_engenharia_cf e importar database/solucoes_engenharia_cf_dump.sql.
3. Confirmar que lib/mysql-connector-j-9.4.0.jar está nas Libraries do projeto.
4. Configurar DB_USUARIO e DB_SENHA como variáveis de ambiente. DB_URL pode ser informada quando a URL padrão não for adequada.
5. Abrir o projeto no NetBeans e executar app.Main.
6. Verificar no Output as listagens, a autenticação e o teste de validação.

12. Critérios de aceitação

- O projeto compila com BUILD SUCCESSFUL.
- A conexão com o MySQL é realizada sem manter a senha no código publicado.
- A autenticação de um usuário ativo com credenciais válidas é concluída.
- As consultas retornam usuários, projetos, auditorias e não conformidades.
- Um projeto sem nome é rejeitado antes de chegar ao repositório.
- As classes de serviço não importam componentes Swing nem executam SQL.
- As implementações JDBC obedecem às interfaces do pacote repository.
- O projeto e sua documentação estão disponíveis no repositório GitHub.

13. Considerações finais

O documento define o escopo e os requisitos da solução utilizada como base para a refatoração. A arquitetura atual mantém o comportamento essencial do sistema desktop e cria uma base reutilizável para a próxima fase de desenvolvimento web.